

# CriticPilot: GenPilot with Planner-Critic Mechanism

课程：计算机图形学A（2026春季·复旦大学） — 项目三

## 摘要

文本到图像生成中的测试时提示优化旨在通过迭代改进提示词，使生成图像更精准地符合用户意图。然而，现有方法如GenPilot采用线性前馈管道，缺乏内部验证机制，面临错误过载与历史污染两大核心瓶颈。受PixelCraft的Planner-Critic架构启发，本文提出CriticPilot，一个对GenPilot的轻量级扩展。CriticPilot在原有优化流程中嵌入错误聚合器、L1局部重试与L3全局重置三个核心模块，赋予系统自校正能力。在10个复杂提示词批次上的实验表明，CriticPilot平均得分提升0.4118分（13.3485 → 13.7604），同时平均评估轮次减少1.5轮，验证了所提机制的有效性与高效性。

代码开源：<https://github.com/zdleeeeeee/PlannerCritic-GenPilot>

## 1. 介绍

文本到图像（T2I）生成领域近年取得了显著进展，但如何使生成图像精准符合用户复杂的文本描述仍是一个开放挑战。测试时提示优化（Test-Time Prompt Optimization）作为一种无需模型微调的轻量级方案，通过迭代改进提示词来提升生成质量，受到广泛关注。

GenPilot (Yuan et al., 2025) 是该领域的代表性工作，利用多模态大语言模型对生成图像进行细粒度错误分析，并据此生成修正建议，在多个benchmark上取得了优异效果。然而，GenPilot采用**线性前馈管道**，缺乏内部验证机制。正如原作者所承认的，基于VQA的错误检测在复杂场景中并不可靠。一旦早期轮次的错误分析出现偏差，后续优化将建立于错误前提之上，导致问题累积且无法自我纠正。

具体而言，GenPilot面临两大核心瓶颈：

- **错误过载 (Error Overload)**：细粒度错误分析可能产生大量冗余错误描述（实测中单个提示词片段可达500余条），导致优化目标发散、关键修正方向被掩盖。
- **历史污染 (History Pollution)**：基于线性累积的优化范式会将早期迭代中产生的错误修改记录持续传递给后续轮次。一旦早期偏离正确方向，错误便会不断累积，最终使分数停滞在局部最优。

受PixelCraft的**Planner-Critic架构** (Microsoft, 2025) 启发——该架构引入批判智能体审查推理轨迹，并在检测到错误时触发回溯——本文提出**CriticPilot**，一个对GenPilot的扩展。CriticPilot在原有优化流程中嵌入Planner-Critic机制，通过错误聚合与分级回溯，赋予系统自校正能力，实现测试时提示优化的高效闭环。实验表明，CriticPilot在10个复杂提示词批次上平均得分提升0.4118分，同时平均评估轮次减少1.5轮。

本文的主要贡献如下：

1. 识别了GenPilot线性优化流程中的错误过载与历史污染两大瓶颈；
2. 提出CriticPilot，一个包含错误聚合器、L1局部重试与L3全局重置的轻量级自校正框架；
3. 在多个复杂提示词批次上验证了所提机制的有效性与高效性。

## 2. 相关工作

---

### 2.1 GenPilot：测试时提示优化

GenPilot (Yuan et al., 2025) 是一个多智能体测试时提示优化系统，通过两个阶段迭代改进提示词：**错误分析阶段**利用VQA和Captioning智能体识别生成图像与提示词之间的语义不一致；**提示词优化阶段**生成多个候选修正，经评分、聚类后选出最优者进入下一轮迭代。

**局限性：**GenPilot的流程是严格线性的，没有任何流程层面的验证机制。正如论文所述，“基于VQA的错误检测在复杂场景中并不可靠”。若错误分析阶段出现偏差，后续优化将建立在错误前提之上，系统无法发现也无法恢复。

### 2.2 PixelCraft：Planner-Critic架构

PixelCraft (Microsoft, 2025) 是一个多智能体视觉推理系统，其核心创新在于**Planner-Critic架构**：批判智能体 (Critic) 负责审查规划器 (Planner) 的推理轨迹，检测工具使用序列中的低效或错误，并在发现问题时触发回溯与分支探索。具体机制包括：（1）**后验精炼**：批判者在讨论完成后审查整个推理轨迹；（2）**回溯与分支**：通过图像记忆存储中间结果，规划器可在遇到矛盾时回退到之前步骤，探索其他推理路径。

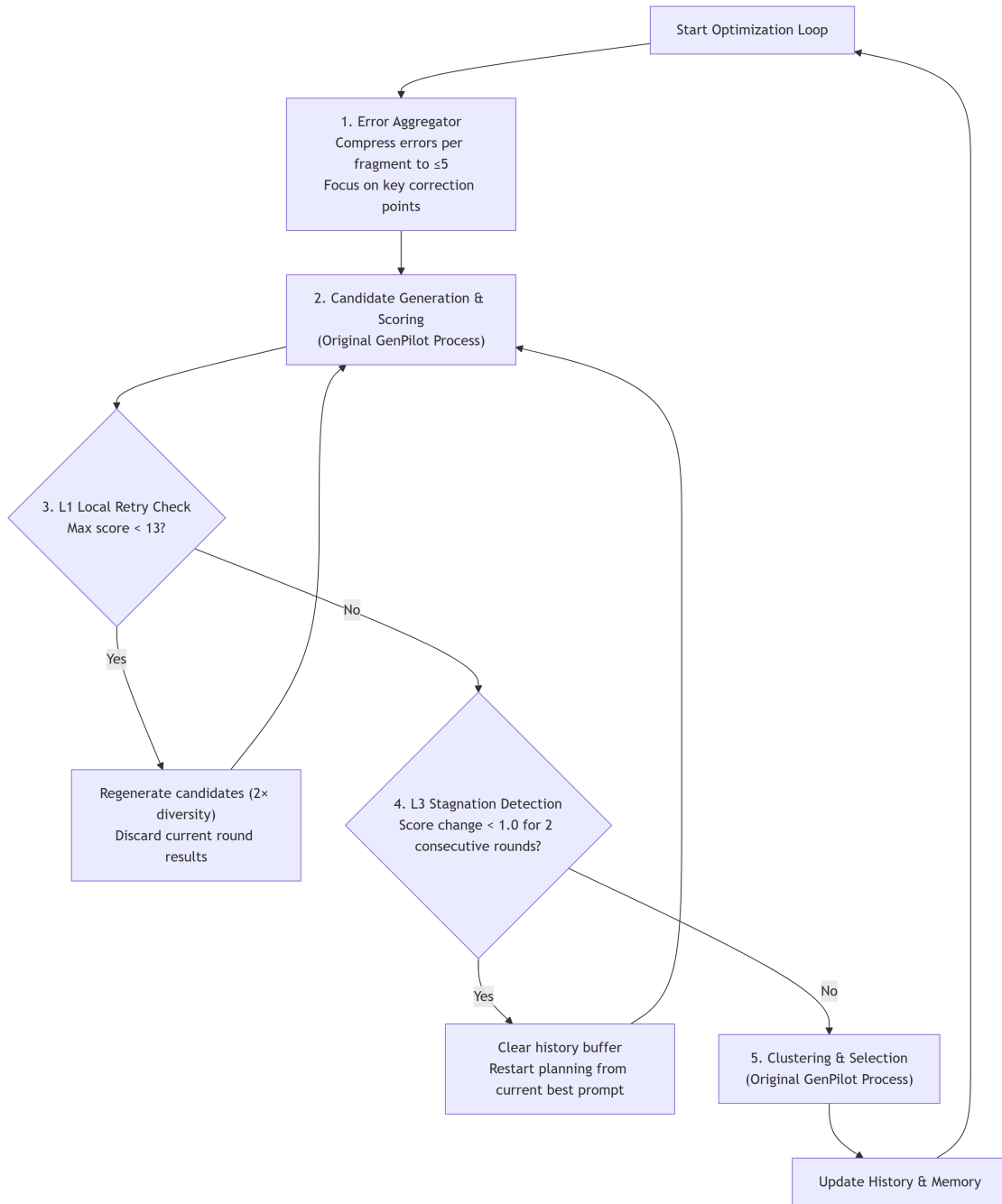
本文借鉴PixelCraft的核心思想，将其适配到提示词优化领域——用错误聚合与分数停滞检测替代显式的Critic LLM，以轻量级方式实现类似的自校正能力。

---

## 3. 方法

CriticPilot在GenPilot的标准优化循环中插入三个核心模块，整体架构如图1所示。

**图1：CriticPilot架构概览**



### 3.1 错误聚合器 (Error Aggregator)

**问题：**细粒度错误分析可能产生大量冗余错误（实测单个片段可达500余条），导致优化目标发散。

**方法：**在每轮优化前，对当前提示词相关的所有错误反馈进行语义聚类，按实体、属性等维度去重，并将每个提示词片段关联的错误数量强约束至不超过5条。

**效果：**将冗余错误压缩一个数量级，聚焦关键修正点，从根本上避免优化信号过载。

## 3.2 L1局部重试 (Local Retry)

**问题：**单轮生成的候选提示词可能整体质量偏低（如因采样随机性）。

**检测条件：**当前轮次所有候选提示词的最高分 < 13（满分15分）。

**动作：**放弃本轮结果，在相同优化上下文中以更高多样性重新生成候选（候选数翻倍）。

**设计依据：**借鉴PixelCraft的“分支”思想，在微观层面进行分支探索，以低成本挽救因采样随机性导致的劣质轮次。

## 3.3 L3全局重置 (Global Reset)

**问题：**历史修改记录（history buffer）可能被错误信息污染，导致优化陷入局部最优。

**检测条件：**连续2轮，最高分提升幅度均小于1.0分。

**动作：**清空所有累积的历史错误修改记录，迫使下一轮从当前最优提示词重新开始规划。

**设计依据：**借鉴PixelCraft的“回溯”概念，打破错误历史对搜索方向的束缚，使系统有机会跳出局部最优。

# 4. 实验

## 4.1 实验设置

**模型与计算平台：**

- 错误分析：Qwen3-VL-8B-Instruct
- 文本到图像生成：FLUX.1 (black-forest-labs/FLUX.1-schnell)
- 所有代码基于GenPilot框架构建

**测试数据：**10个测试批次，每批包含若干精心构造的复杂提示词，覆盖物体计数、空间关系、属性绑定等典型失败模式。

**基线 (Baseline)：**标准GenPilot优化流程，不含任何CriticPilot模块。

**评估指标：**

- 生成质量：统一的多模态评分模型自动评定，满分15分
- 计算开销：实际调用图像生成与评分的轮次 (Scored Rounds)
- 机制活跃度：L1局部重试与L3全局重置的触发次数

## 4.2 总体评分表现

10个批次的汇总数据显示，CriticPilot平均得分达到13.7604，基线为13.3485，提升**+0.4118分**。除Batch6基本持平 ( $\Delta = -0.033$ ) 外，其余9个批次均录得正向收益，其中Batch4增益最显著， $\Delta$ 达+0.841，证明该机制具有良好的泛化性。

表1：总体评分表现（10批次）

| Batch   | Baseline avg | Critic avg | Delta  | L1   | L3  | Baseline scored rounds | Critic scored rounds | Scored rounds difference |
|---------|--------------|------------|--------|------|-----|------------------------|----------------------|--------------------------|
| 1       | 13.5         | 13.605     | 0.105  | 13   | 4   | 28                     | 38                   | 10                       |
| 2       | 13.87        | 14.316     | 0.446  | 3    | 1   | 23                     | 19                   | -4                       |
| 3       | 13.371       | 13.871     | 0.5    | 8    | 1   | 35                     | 31                   | -4                       |
| 4       | 12.288       | 13.13      | 0.841  | 38   | 1   | 59                     | 54                   | -5                       |
| 5       | 12.69        | 13.39      | 0.7    | 20   | 4   | 42                     | 41                   | -1                       |
| 6       | 13.85        | 13.817     | -0.033 | 9    | 5   | 60                     | 60                   | 0                        |
| 7       | 13.211       | 13.628     | 0.417  | 17   | 3   | 38                     | 43                   | 5                        |
| 8       | 13.588       | 14.045     | 0.457  | 11   | 3   | 51                     | 44                   | -7                       |
| 9       | 13.242       | 13.469     | 0.227  | 22   | 5   | 62                     | 64                   | 2                        |
| 10      | 13.875       | 14.333     | 0.458  | 1    | 1   | 32                     | 21                   | -11                      |
| Overall | 13.3485      | 13.7604    | 0.4118 | 14.2 | 2.8 | 43                     | 41.5                 | -1.5                     |

图2：基线vsCriticPilot图片质量平均分对比

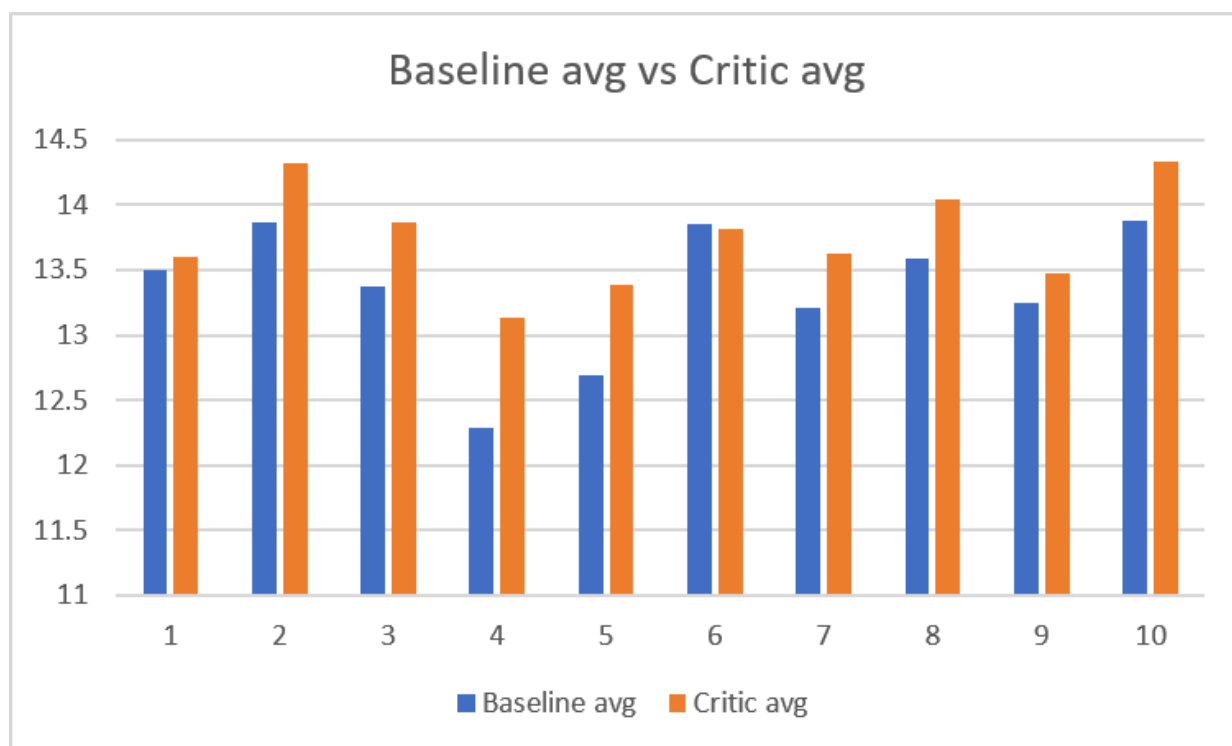
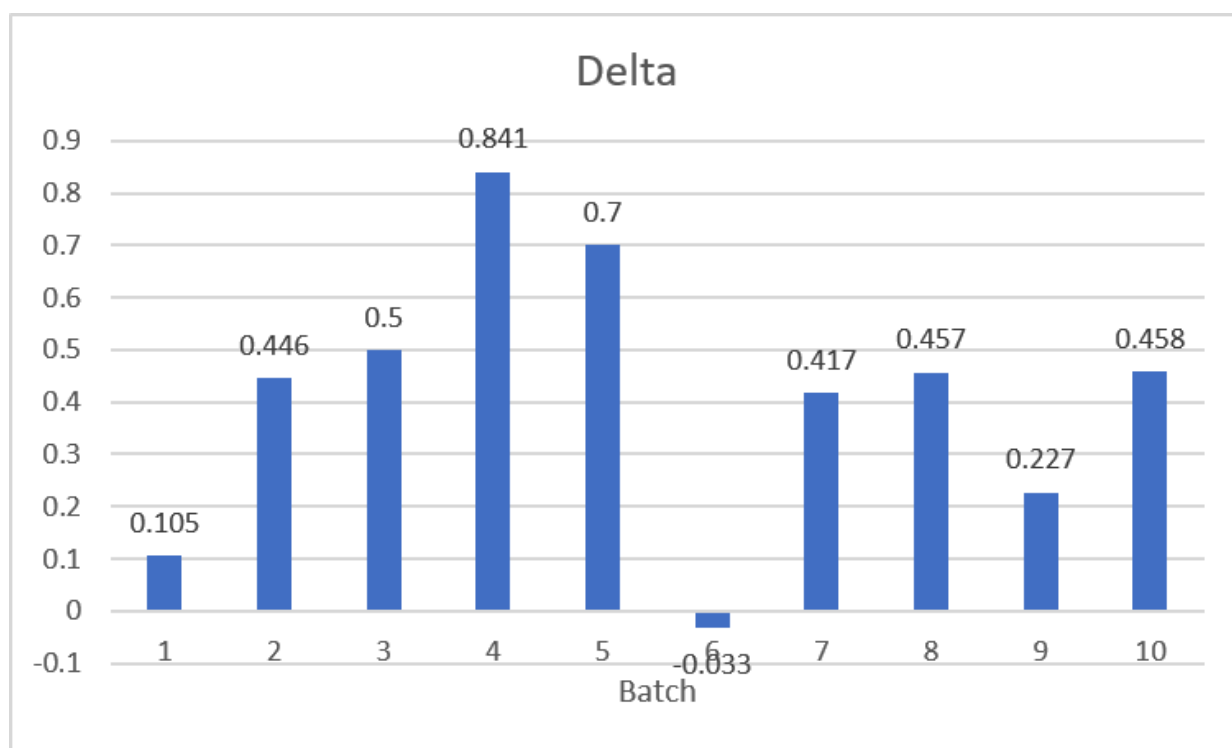


图3：CriticPilot相对于基线图片质量平均分差值



### 4.3 计算开销与效率

通常情况下，重试与重置机制会带来额外计算开销。然而实验数据表明：CriticPilot的平均评估轮次为41.5，低于基线的43，平均减少**1.5轮**。原因在于L1与L3起到了“剪枝”与“导航”的双重作用：局部重试避免了无效低分轮次被计入并浪费后续优化；全局重置则果断终止停滞路径，将计算资源重新投向更宽广的搜索空间。

图4：基线vsCriticPilot平均评估轮次

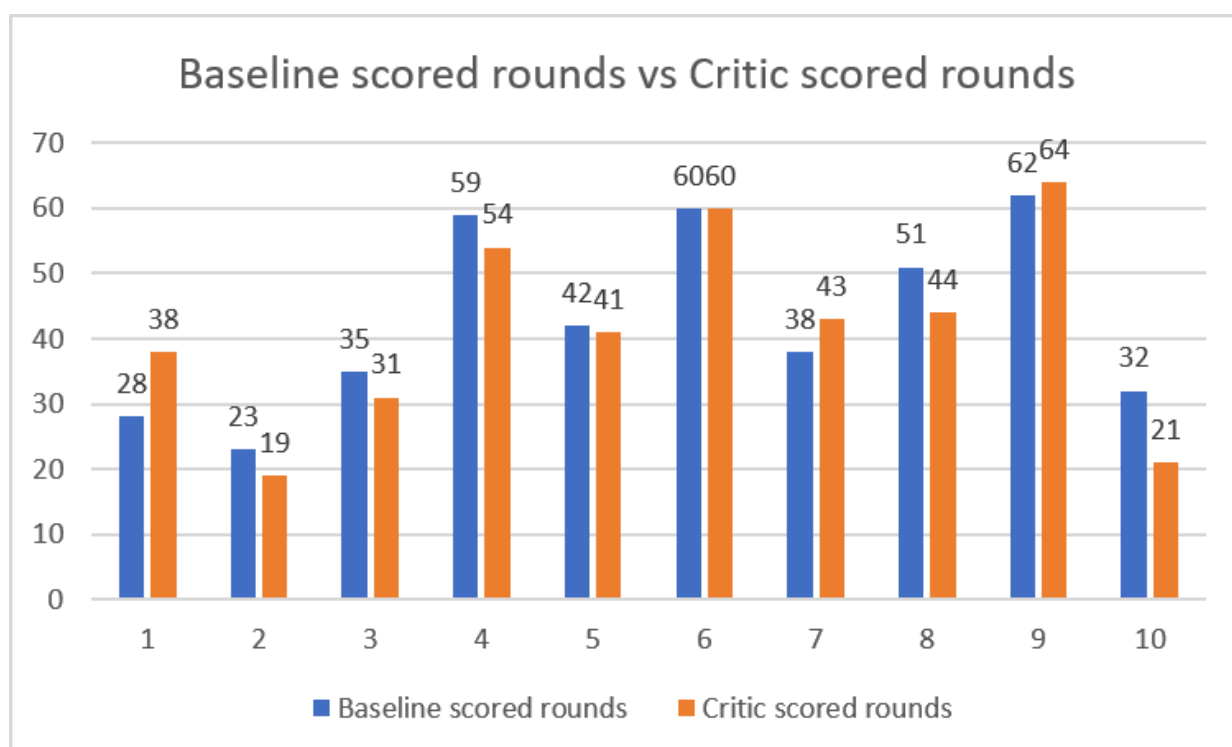
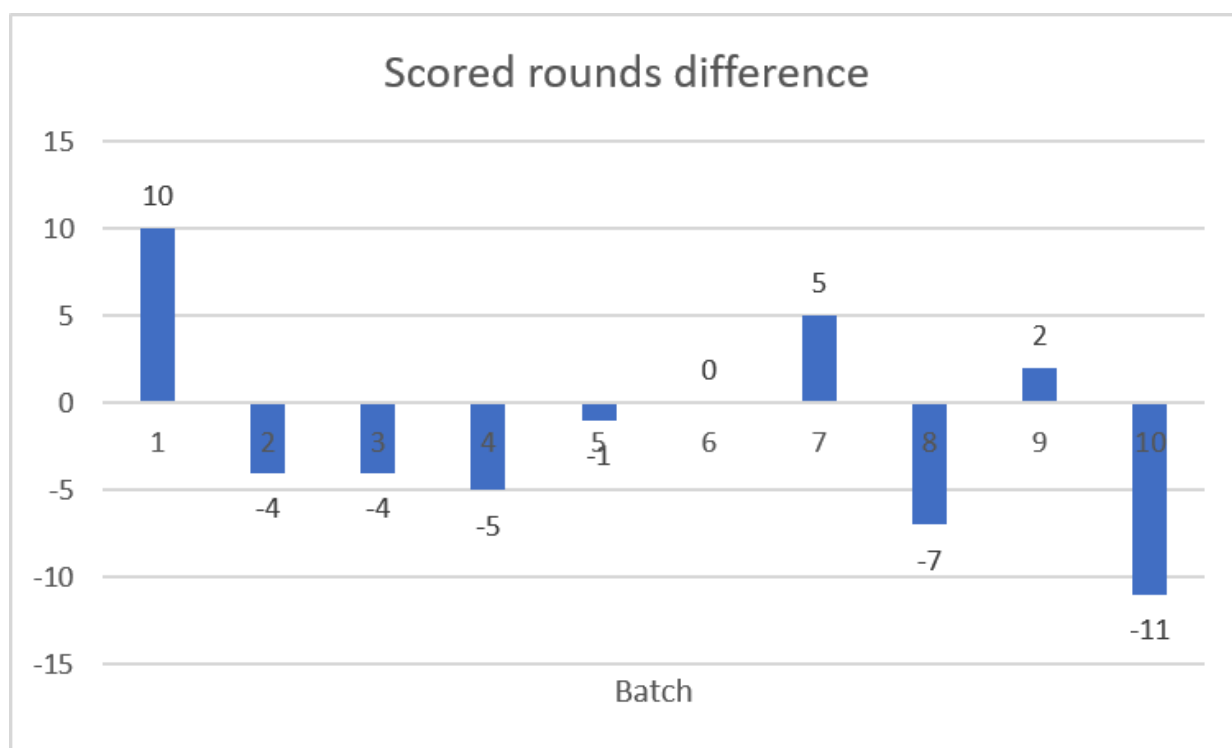


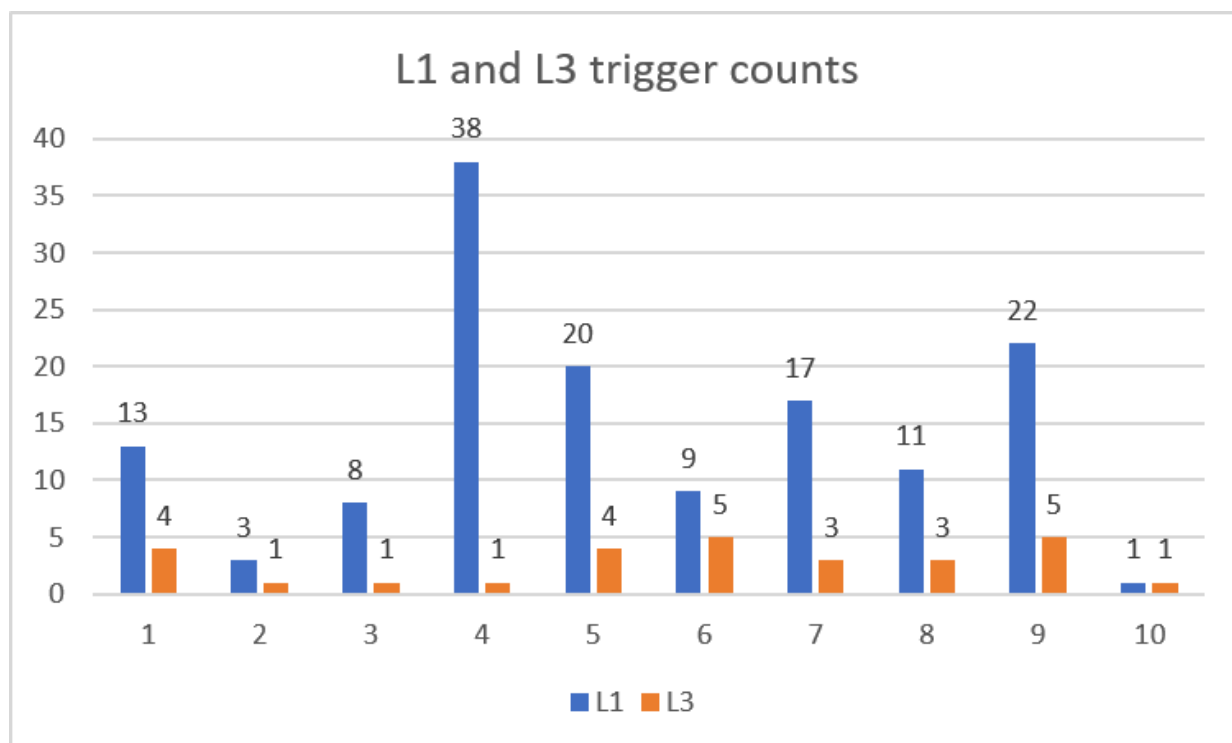
图5：CriticPilot相对于基线平均评估轮次差值



## 4.4 机制触发频率

统计每个批次的平均触发次数：L1触发14.2次，L3触发2.8次。L1的高频触发说明原始优化过程中经常产生低质量候选，局部重试提供了一种廉价的自救手段。L3触发频次较低但意义重大——每次全局重置都使系统挣脱错误历史的束缚，获得重新探索并找到更优解的机会（如Batch4中L3触发后分数显著跃升）。这一频率印证了停滞并非偶发现象，而是线性优化固有的弊端。

图6：L1和L3触发次数



## 4.5 案例分析

### 成功案例A：精确计数约束（Batch1）

**原始提示词（节选）：**“A cabbage field with exactly 8 cabbages, each cabbage has dewdrops, and the field is covered with light mist.”

**Baseline问题：**生成的卷心菜数量经常偏离8，且露珠、薄雾时常丢失，得分在12分左右震荡。

**机制作用：**错误聚合器将错误聚焦于“计数”与“可见度”。通过L1/L3的反复介入，系统演化出极强的硬性约束，例如：“... featuring exactly eight distinct cabbages, no more and no fewer. Every cabbage must display visible dew droplets. A light veil of mist hangs over the entire field ...”。最高分从13跃升至15（满分），优化上限被成功打破。

### 成功案例B：消除物体幻觉（Batch4）

**原始提示词：**“A silver bracelet resting on a chessboard, the bracelet is compact, not larger than one square.”

**Baseline问题：**产生严重幻觉，将“银手镯”错误生成为带管状结构的“听诊器”，且尺寸巨大横跨多个棋盘格。

**机制作用：**错误聚合锁定“错误物体类型”和“尺寸失控”两大核心问题。系统生成了高度针对性的负面提示词（明确排除听诊器部件，如“no chestpiece, no tubes”）以及利用场景参照物的空间约束（“the bracelet must be no larger than one chessboard square”）。稳定消除了幻觉，大幅提高了生成质量下限，最低分轮次显著减少，因此Batch4的 $\Delta$ 高达+0.841。

### 异常值剖析：Batch6

在所有10个批次中，Batch6是唯一 $\Delta$ 为负（-0.033）的批次。两者最高分均为满分15，评估轮次也完全相同（60轮），差异仅来源于候选质量的微小波动。

**案例细节：**提示词要求“A vintage fan with a rounded base placed beside an antique knife on a wooden table”。Baseline表现已较强，但Critic为强化风扇与刀的区分，采取了过于激进的排他性描述（如“loose circular fan head / no pedestal base”），这反而偏离了原始提示中关于“rounded base”的固有要求，导致部分轮次分数轻微下降。

**根因与启示：**当Baseline已逼近性能上限，或原始提示本身存在语义歧义时，Critic的强约束可能导致过度修正。该边界条件揭示：未来可引入自适应干预策略，在优化后期适当降低重试与重置的激进程度，以避免因过度拟合错误信号而引入新矛盾。总体而言，Batch6更应被视为“Critic与Baseline基本持平”，而非显著失败。

## 5. 结论

本文提出CriticPilot，一个对GenPilot的轻量级扩展，通过嵌入Planner-Critic机制解决线性优化流程中的错误过载与历史污染问题。实验验证了CriticPilot的核心价值：

- 增效不增本：**在平均评估轮次减少1.5轮的前提下，平均得分提升0.4118分，证明“质量控制与错误回溯”比“单纯堆叠迭代轮数”更为重要。
- 上下限双提升：**L1局部重试抑制低分轮次，稳固性能下限；L3全局重置清除历史污染，拔高分数上限。



3. **高鲁棒性**：在90%的测试批次中获正向收益，仅在一个存在歧义的场景中出现可接受的微小回退，未发生崩溃。

综上，CriticPilot作为一种架构改动极小、无额外LLM调用的轻量级方案，以较高的效费比在一定程度上解决了提示词自动优化中的算力浪费与停滞难题。未来工作将探索自适应干预策略，在优化后期适当降低重试与重置的激进程度，以避免因过度修正而引入新矛盾。

---

## 参考文献

1. Yuan, Y., et al. "GenPilot: A Multi-Agent System for Test-Time Prompt Optimization in Image Generation." Findings of EMNLP 2025.
2. Microsoft Research. "PixelCraft: Multi-Agent Visual Reasoning with Planner-Critic Architecture." 2025.
3. GenPilot开源代码. <https://github.com/27yw/GenPilot>

---

## 附录A：分工

- **李泽栋 23307130271**：设计实验思路和实验计划，复现baseline，搭建基本项目代码框架，编写实验报告
- **颜皓鹏 24300240240**：实现CriticPilot代码，执行测试，记录数据
- **沈子轩 24300240213**：分析数据，整理结果

---

## 附录B：系统实现与复现指南

### 项目结构

```
1  PlannerCritic-GenPilot
2  ├── .env.example
3  ├── .gitignore
4  ├── .python-version
5  ├── LICENSE
6  ├── README.md
7  ├── data/
8  |   ├── original_prompts.txt
9  |   └── original_prompts_299.txt
10 ├── genpilot/
11 ├── pyproject.toml
12 ├── run_baseline.py
13 ├── tests/
14 |   ├── test_compare_metrics.py
15 |   ├── test_error_taxonomy.py
16 |   └── test_scorer_json.py
17 └── uv.lock
```

## 安装与环境配置

```
1 # 克隆仓库
2 git clone https://github.com/zdleeeee/PlannerCritic-GenPilot.git
3 cd PlannerCritic-GenPilot
4
5 # 复制环境配置文件
6 cp .env.example .env
7 cp genpilot/error_analysis_pipeline.sh.example
  genpilot/error_analysis_pipeline.sh
8
9 # 安装依赖（使用uv）
10 uv sync
11
12 # 下载文生图模型（如需）
13 hf download black-forest-labs/FLUX.1-schnell --local-dir <本地下载路径>
```

**注意：**我们将genpilot环境中的 `mk1-service` 包从2.4.0升级至2.5.2，以解决其与Python 3.12不兼容的问题。

## 运行流程

```
1 | python run_baseline.py
```

该脚本同时运行基线GenPilot与CriticPilot。

## 测试

```
1 | tests/test_error_taxonomy.py
2 | tests/test_scorer_json.py
3 | tests/test_compare_metrics.py
```